Linking and Running C Program

From Compilation to Execution

After compilation, the next step is linking the program to create an executable file that can run on your system!

The Linking Process 🔗

After compilation, the next step is linking the program. Compilation produces a file with an extension .obj.

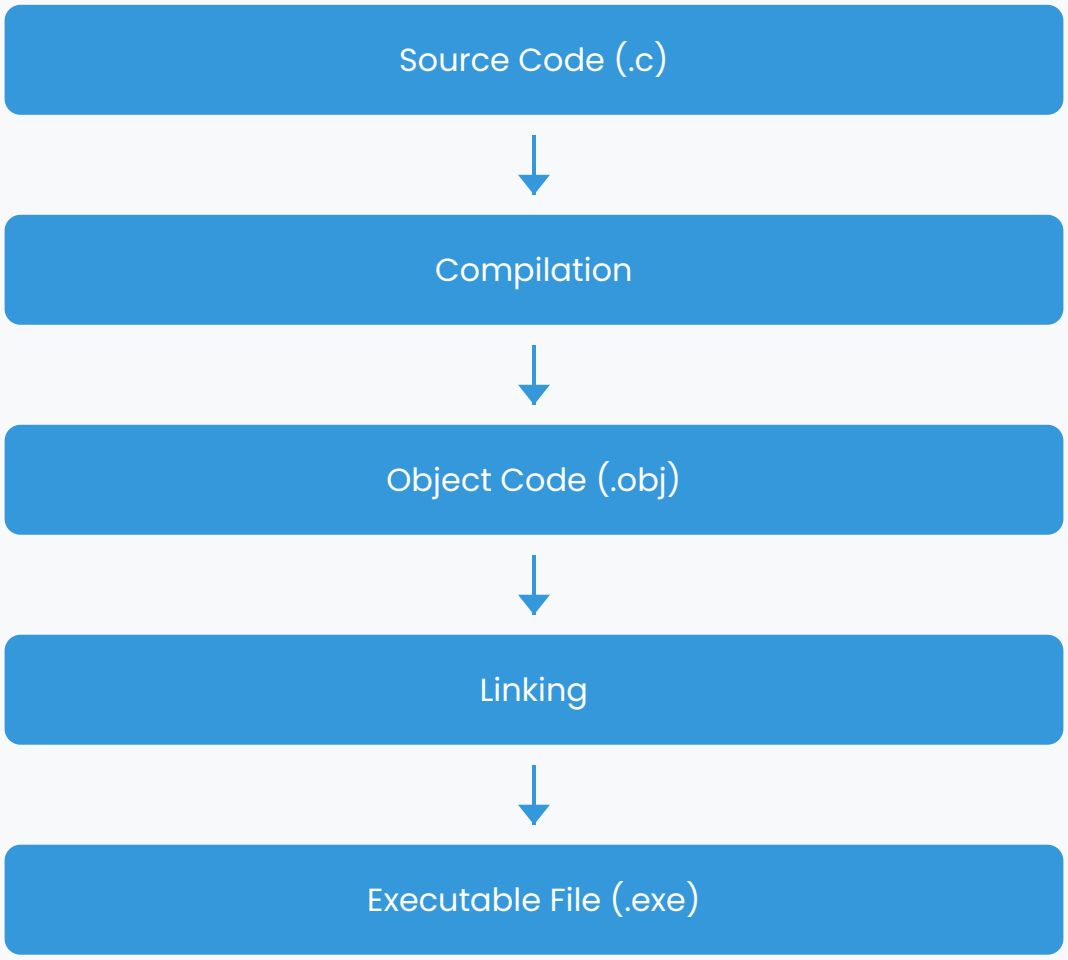
Now this .obj file cannot be executed since it contains calls to functions defined in the standard library (header files) of C language. These functions have to be linked with the code you wrote.

C comes with a standard library that provides functions that perform most commonly needed tasks. When you call a function that is not the part of the program you wrote, C remembers its name.

Later the linker combines the code you wrote with the object code already found in the standard library. This process is called linking.

💡 What is a Linker?

In other words, Linker is a program that links separately compiled functions together into one program. It combines the functions in the standard C library with the code that you wrote. The output of the linker is an executable program i.e., a file with an extension .exe.



Running Program through Menu 🖱️

When we are working with TurboC in DOS environment, the menu in the GUI that pops up when we execute the executable file of TurboC contains several options for executing the program:

(i) Link, after compiling

This option links the already compiled object code with the standard library functions to create an executable file.

(ii) Make, compiles as well as links

This option performs both compilation and linking in one step, creating an executable file directly from the source code.

(iii) Run

This option compiles, links, and executes the program, showing the output on the user screen.

💡 Viewing Output

All these options create an executable file and when these options are used we also get the output on user screen. To see the output we have to shift to user screen window.

Running Executable File ▶️

An .exe file produced can be directly executed.

UNIX Environment

UNIX also includes a very useful program called make. Make allows very complicated programs to be compiled quickly, by reference to a configuration file (usually called makefile).

If your C program is a single file, you can usually use make by simply typing:

Command

```
make testprog
```

This will compile testprog.c as well as link your program with the standard library so that you can use the standard library functions such as printf and put the executable code in testprog.

DOS Environment

In case of DOS environment, the options provided above produce an executable file and this file can be directly executed from the DOS prompt just by typing its name without the extension.

That is if the name of the program is test.c, after compiling and linking the new file produced is test.exe only if compilation and linking is successful.

This can be executed as:

Command

```
c>test
```

Linker Errors ❌

If a program contains syntax errors then the program does not compile, but it may happen that the program compiles successfully but we are unable to get the executable file, this happens when there are certain linker errors in the program.

For example, the object code of certain standard library function is not present in the standard C library; the definition for this function is present in the header file that is why we do not get a compiler error.

Such kinds of errors are called linker errors. The executable file would be created successfully only if these linker errors are corrected.

Common Linker Errors

- Undefined reference to function
- Multiple definitions of the same function
- Missing library files
- Incorrect function signatures

💡 Key Difference

Unlike syntax errors that prevent compilation, linker errors occur after successful compilation but prevent the creation of an executable file.

Logical and Runtime Errors 🐛

After the program is compiled and linked successfully we execute the program. Now there are three possibilities:

Program executes and we get correct results

Program executes and we get wrong results

Program does not execute completely and aborts in between

Logical Error

🧠 In the second case, we get wrong results; it means that there is some logical mistake in our program. This kind of error is known as logical error. This error is the most difficult to correct.

Runtime Error

⚠️ The third kind of error is only detected during execution. Such errors are known as run time errors. These errors do not produce the result at all, the program execution stops in between and the run time error message is flashed on the screen.

Example of Logical Error 🧮

Example: Create a C program to compute the average of three integers.

```c
/* Program to compute average of three numbers */
#include<stdio.h>
main( )
{
    int a,b,c,sum,avg;
    a=10; b=5; c=20;
    sum = a+b+c; avg = sum / 3;
    printf("The average is %d\n", avg);
}
```

Output:

The average is 8.

💡 Explanation of the Logical Error

The exact value of average is 8.33 and the output we got is 8. So we are not getting the actual result, but a rounded off result. This is due to the logical error.

We have declared variable avg as an integer but the average calculated is a real number, therefore only the integer part is stored in

avg. Such kinds of errors which are not detected by the compiler or the linker are known as logical errors.

Example of Runtime Error ⚠️

Example: Write a program to divide a sum of two numbers by their difference

```c
/* Program to divide a sum of two numbers by their difference*/
#include <stdio.h>
main( )
{
    int a,b; float c;
    a=10; b=10;
    c = (a+b) / (a-b);
    printf("The value of the result is %f\n",c);
}
```

Runtime Error:

The above program will compile and link successfully, it will execute till the first printf statement and we will get the message in this statement, as soon as the next statement is executed we get a runtime error of "Divide by zero" and the program halts.

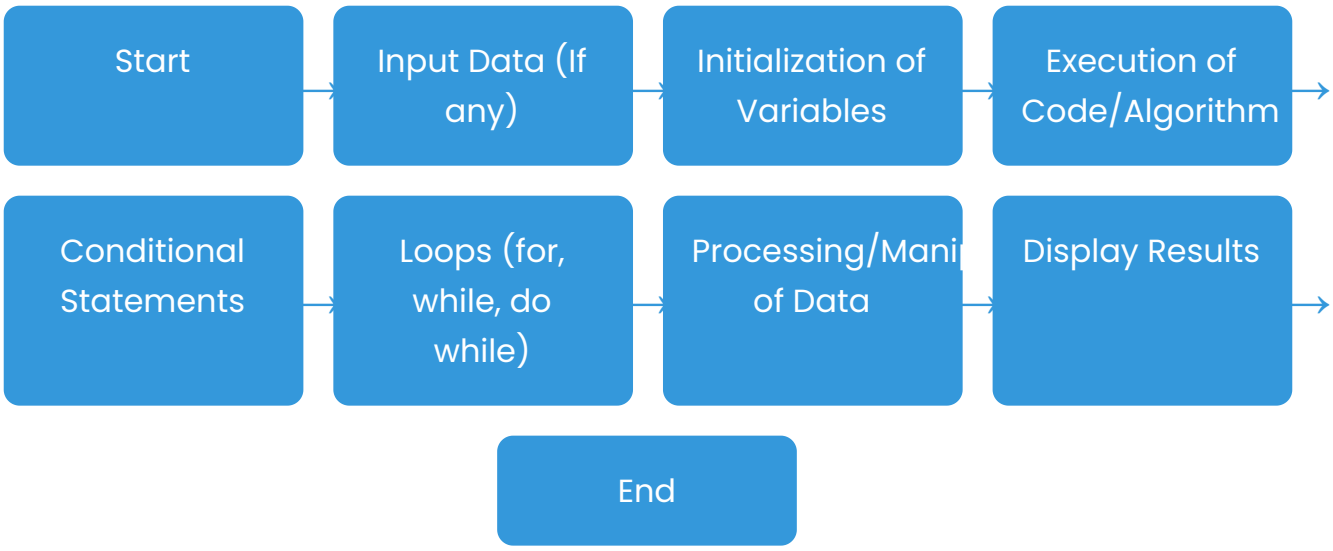
💡 Explanation of the Runtime Error

In this example, when a and b are both 10, the expression (a-b) becomes 0, leading to a division by zero situation. This error is not

caught during compilation or linking but causes the program to crash during execution.

Diagrammatic Illustration 📊

The diagrammatic depiction of the program execution process is shown below. A flowchart representing the execution process of a C program is generally composed of a series of stages.



💡 Program Flow Explanation

This flowchart demonstrates the general flow of a C program. It starts with data input (if necessary), followed by initializing variables, executing the main code or algorithm, applying conditional statements or loops as needed, calling functions (if used), processing data, displaying results, and finally, ending the program. This flow can

vary depending on the specific logic and structure of the C program being executed.

Conclusion 🎓

You learnt about a program and a programming language in this unit. Languages can now be classified as high-level or low-level. You may now define C and its characteristics. You have investigated C's emergence.

You've seen how C differs from other High-Level languages by being a middle-level language. The benefits of using high-level language over low-level language are highlighted.

You've seen how to turn an algorithm and a flowchart into a C program. In both the UNIX and DOS environments, we addressed the process of creating and saving a C program in a file.

You've learned how to compile and run a C program in both UNIX and DOS environments. We also spoke about the many sorts of problems that occur along the process, including as syntax errors, semantic errors, logical errors, linker errors, and runtime issues. You've also learned how to fix these mistakes.

Ready for Next Steps

Simple C programs employing simple arithmetic operators and the printf() function are now possible. Now that we've covered the fundamentals, we're ready to dive into the C language in depth in the next units.